TECNOLÓGICO NACIONAL DE MÉXICO

Campus Tuxtla Gutiérrez

Mini Proyecto 2
Unidad 3

Roles, Policies y Bitácoras en Laravel

Documento integrado de evidencias

Julio Alejandro Medina Cervantes

Rodrigo de Jesús Interiano Zúñiga

Judith Marroquín

27 de octubre de 2026

REPORTE INTEGRADO

Objetivo y organización

Se extendió el E-commerce con relaciones Eloquent, autenticación manual, roles, Policies, FormRequest, Seeders, Factory y tres bitácoras independientes. Las capturas de este documento fueron generadas a partir de la versión local terminada y de su base SQLite.

Distribución del trabajo

Integrante	Responsabilidades
Rodri Usuarios y seguridad	Modelo Usuario, autenticación, provider, roles, Policy, Factory y bitácora de autenticación.
Judith Productos y categorías	CRUD, relaciones, tabla pivote, FormRequest, Policies, Seeders y bitácora de productos.
Julio Ventas e integración	CRUD de ventas, cliente y vendedor, VentaPolicy, evento de venta, configuración de logs, relación de categorías por usuario e integración de módulos.

Contenido

1	Modelo Usuario y autenticación	6	Relación Venta
2	Login manual	7	Validación con FormRequest
3	Policies y Gates	8	Seeders y Factory
4	Relación Usuario–Producto	9	Bitácoras del sistema
5	Relación Producto–Categoría	10	Migraciones y llaves foráneas
11	Usuario → Productos → Categorías y anexo de rutas		

Pruebas realizadas: 45 pruebas automatizadas correctas, 156 aserciones, migración limpia con Seeders y recorrido completo en navegador para los tres roles.

EVIDENCIA 1 · SISTEMA

Usuarios registrados

Responsable principal: Rodri

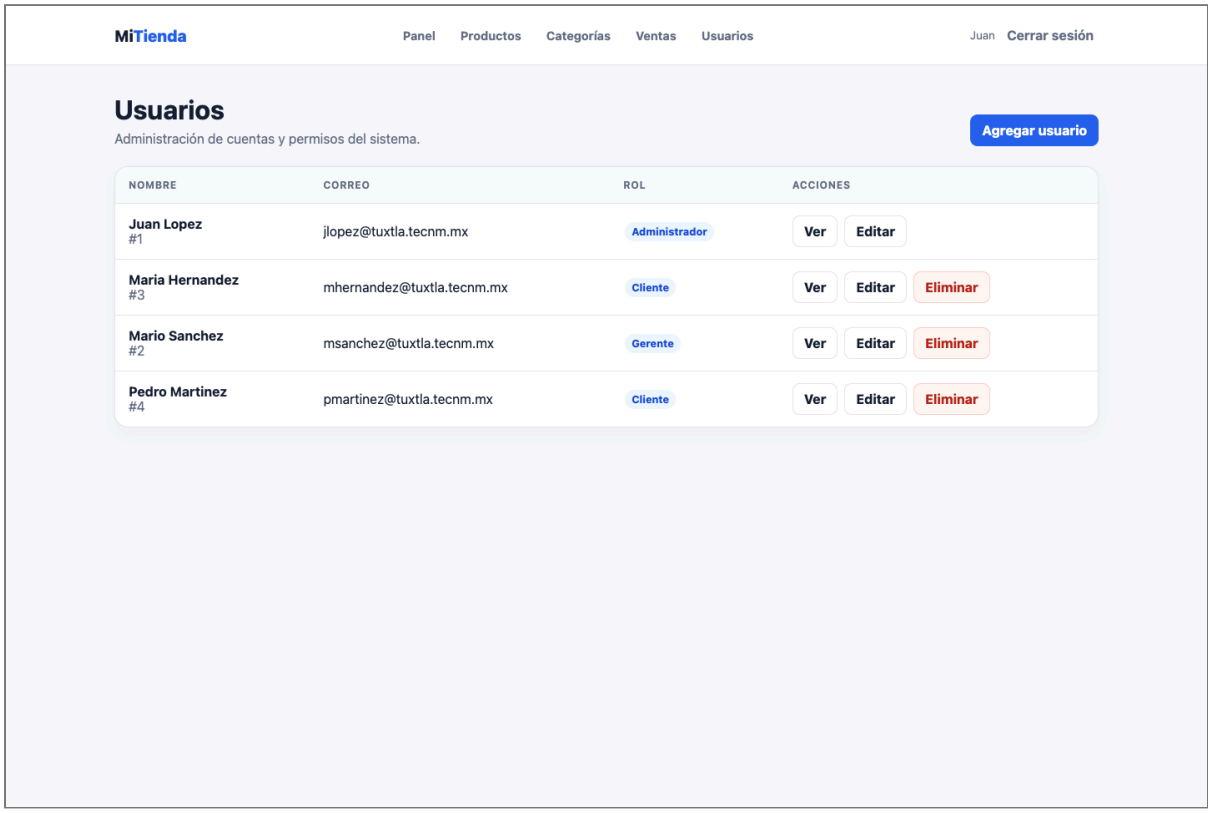


Figura 1. Listado real de usuarios con nombre, correo y rol.

Descripción breve:

El sistema utiliza la tabla **usuarios** y distingue los roles Administrador, Gerente y Cliente. Desde este módulo el administrador puede crear, consultar, editar y eliminar cuentas, mientras que el gerente únicamente ve y edita clientes.

EVIDENCIA 1 · SISTEMA

Login y panel del administrador

Responsable principal: Rodri

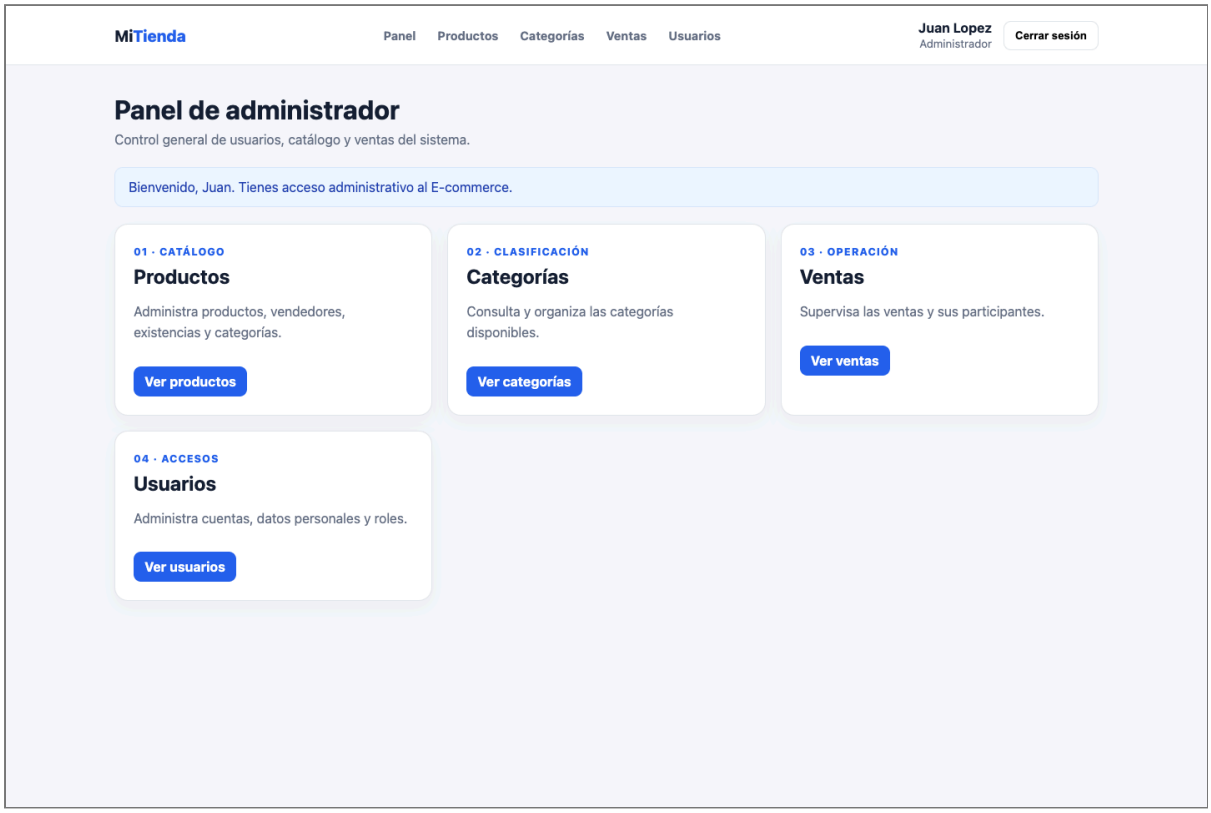


Figura 2. Acceso correcto de Juan Lopez y menú completo del sistema.

Descripción breve:

Después de validar las credenciales, el sistema redirige al panel correspondiente al rol. El menú concentra Usuarios, Productos, Categorías y Ventas para que todas las operaciones se realicen desde la interfaz.

EVIDENCIA 1 · CÓDIGO

Usuario y provider de autenticación

Responsable principal: Rodri

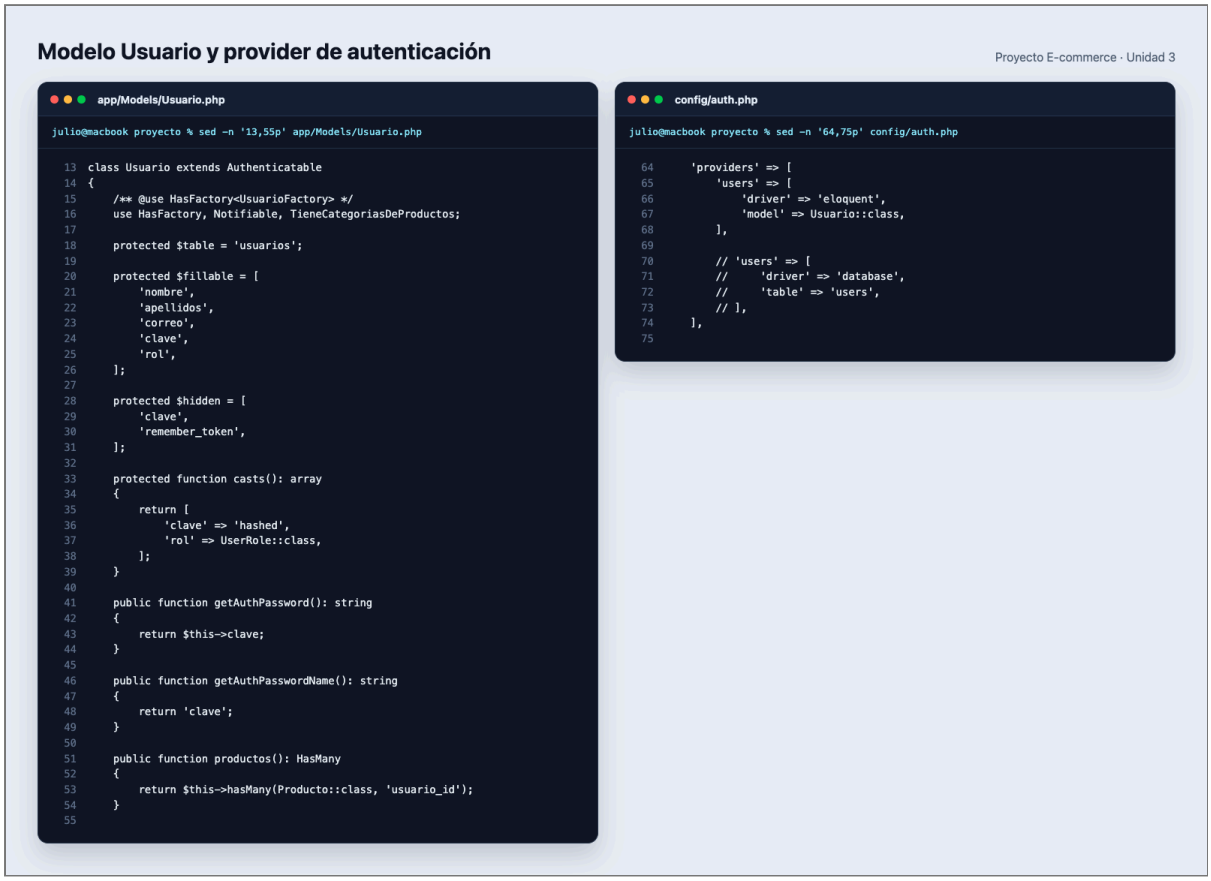


Figura 3. Modelo Authenticatable, campos protegidos y provider configurado con Usuario.

Descripción breve:

Usuario hereda de Authenticatable para participar en el guard web. El provider de **config/auth.php** apunta a este modelo y los métodos de contraseña indican que la credencial se almacena en la columna **clave**.

EVIDENCIA 2 · SISTEMA

Login incorrecto

Responsable principal: Rodri

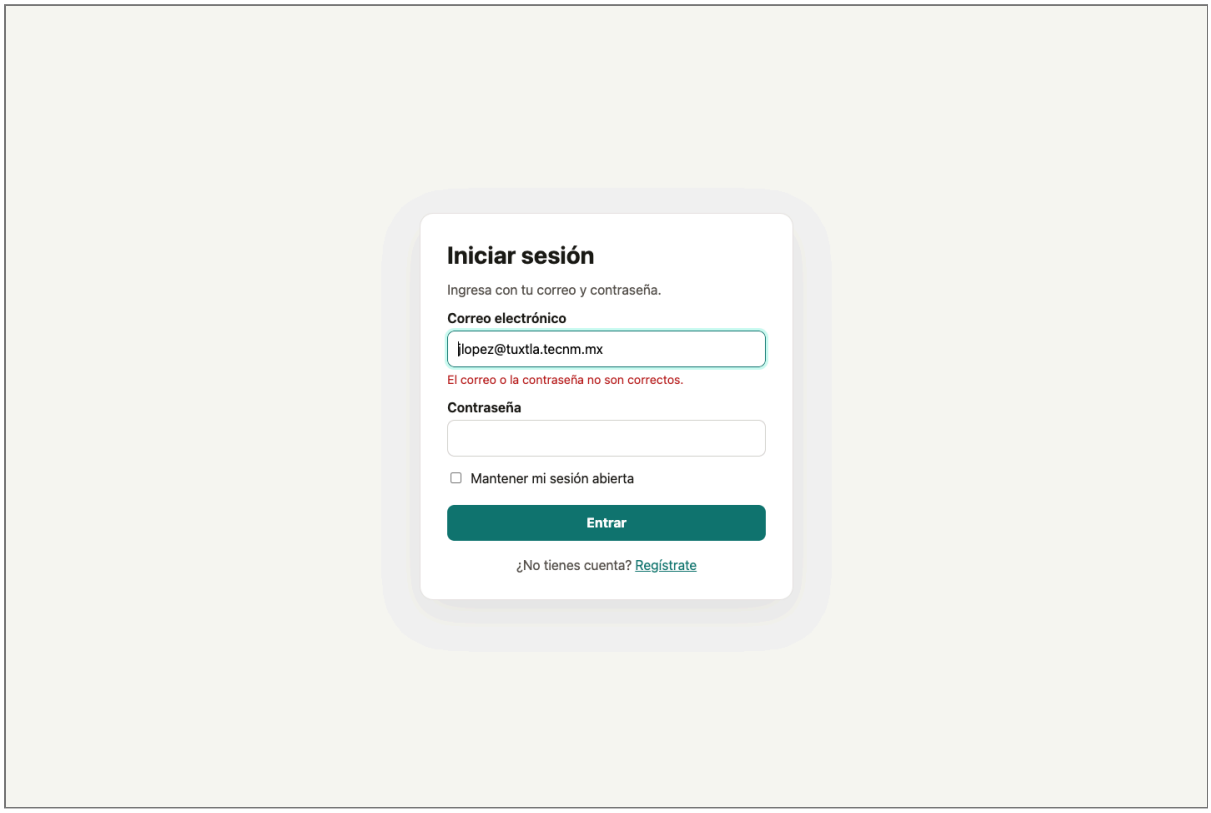


Figura 4. Respuesta del formulario ante una contraseña incorrecta.

Descripción breve:

Cuando las credenciales no coinciden, se mantiene al usuario en el formulario y se presenta un mensaje general que no revela si el correo existe. El intento también se registra en **autenticacion.log**.

EVIDENCIA 2 · CÓDIGO

Auth::attempt y cierre de sesión

Responsable principal: Rodri

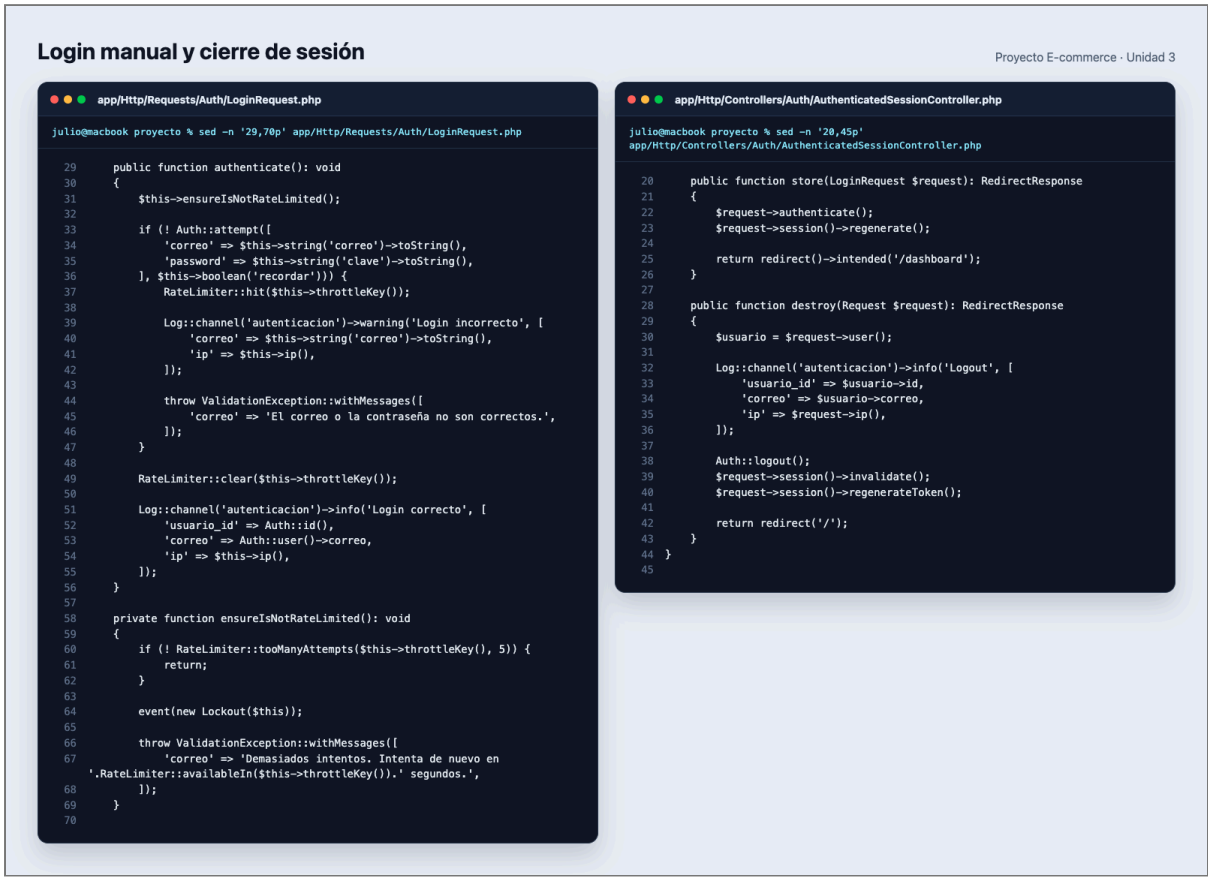


Figura 5. Autenticación manual, limitación de intentos, regeneración e invalidación de sesión.

Descripción breve:

El FormRequest ejecuta **Auth::attempt()** con correo y clave, aplica rate limiting y registra el resultado. En un acceso correcto el controlador regenera la sesión; al salir registra el evento, invalida la sesión y renueva el token CSRF.

EVIDENCIA 3 · SISTEMA

Acceso denegado por rol

Responsables: Rodri, Judith y Julio

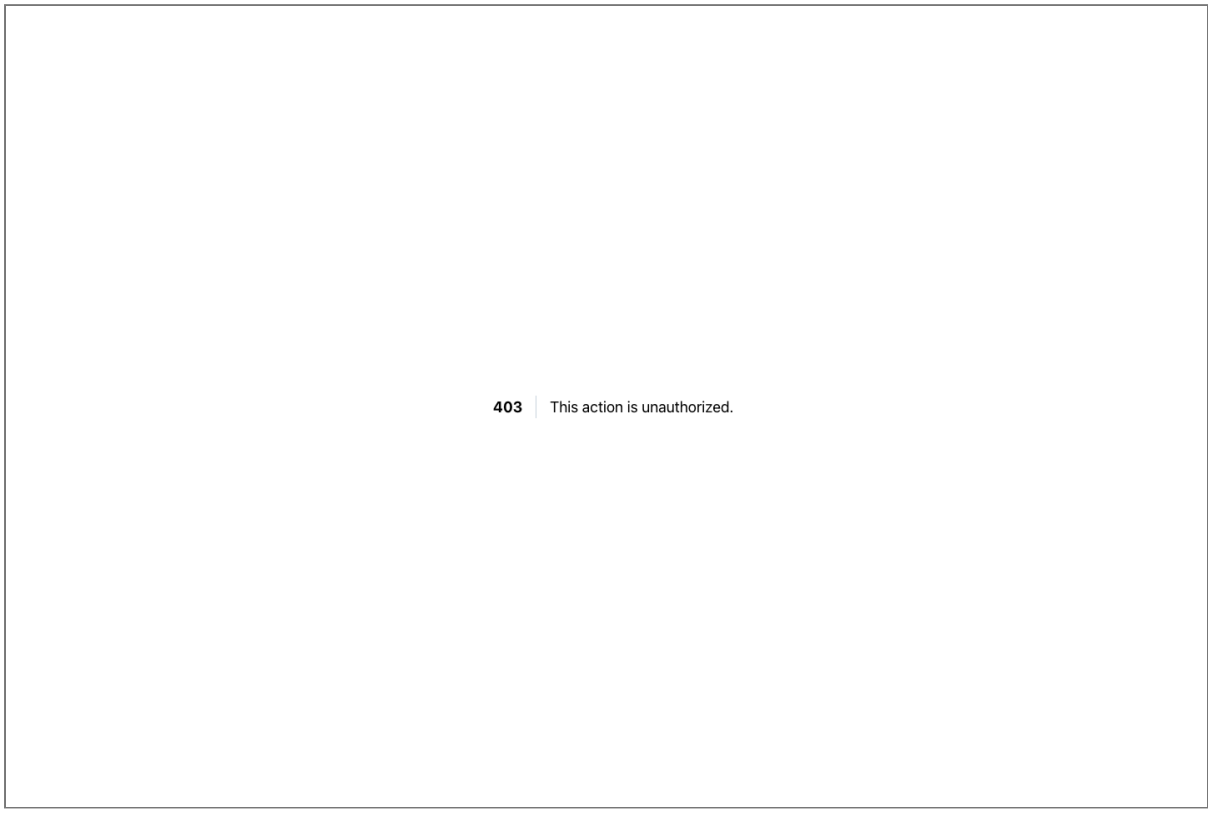


Figura 6. La cliente Maria Hernandez intenta abrir la edición de un producto y recibe respuesta 403.

Descripción breve:

El cliente puede consultar y comprar productos, pero no administrarlos. Al solicitar una acción fuera de sus permisos, Laravel detiene el flujo antes de modificar datos y devuelve **403 Unauthorized**.

EVIDENCIA 3 · CÓDIGO

Policy de productos y Gate::authorize

Responsables: Rodri, Judith y Julio

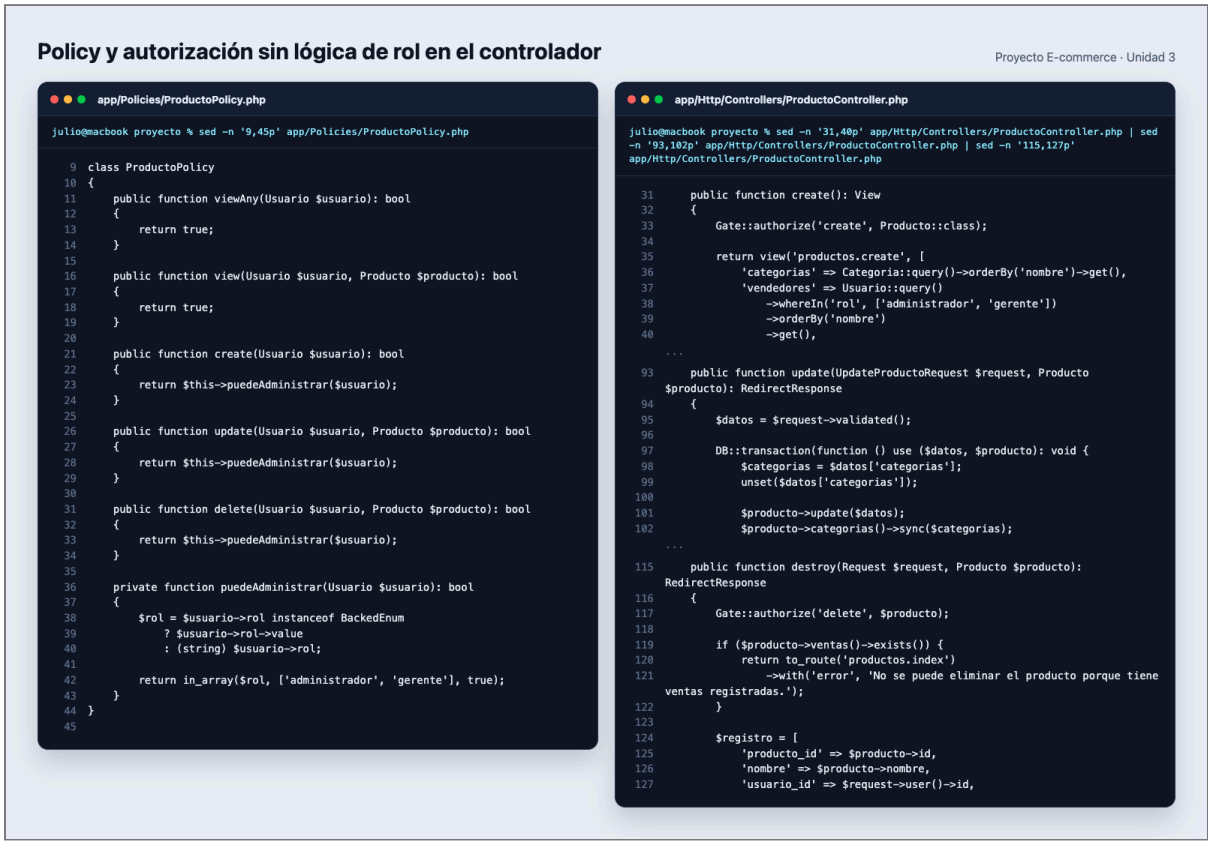


Figura 7. Reglas centralizadas en ProductoPolicy y autorización aplicada desde el controlador.

Descripción breve:

Las decisiones de acceso viven en las Policies. El controlador llama a **Gate::authorize()** para cada operación y se limita a coordinar el CRUD; no contiene condiciones manuales para permitir o negar acciones por rol.

EVIDENCIA 4 · SISTEMA

Productos y sus vendedores

Responsables: Judith y Rodri

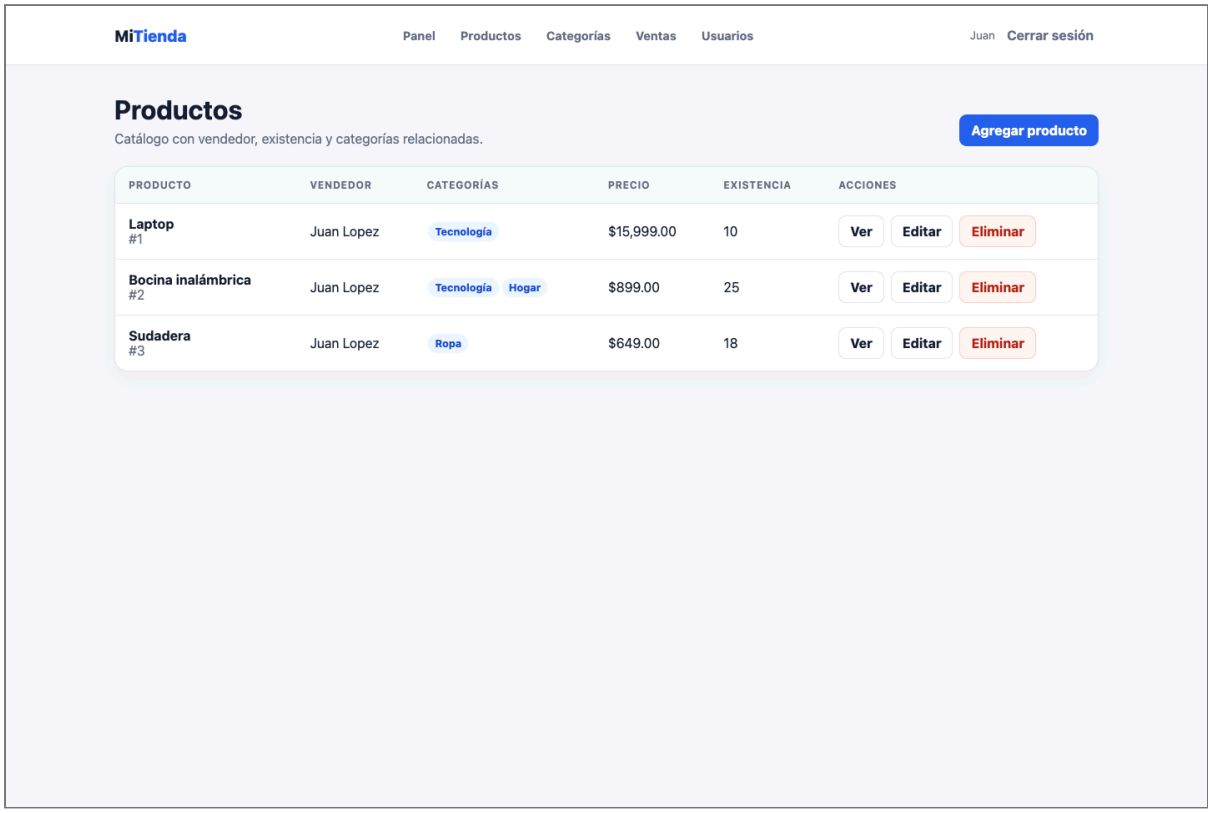


Figura 8. Catálogo real con vendedor, categorías, precio y existencia.

Descripción breve:

Cada producto muestra al usuario que lo vende. La relación se resuelve mediante **usuario_id** y se carga junto con las categorías para evitar consultas repetidas en la vista.

EVIDENCIA 4 · CÓDIGO

belongsTo y hasMany

Responsables: Judith y Rodri

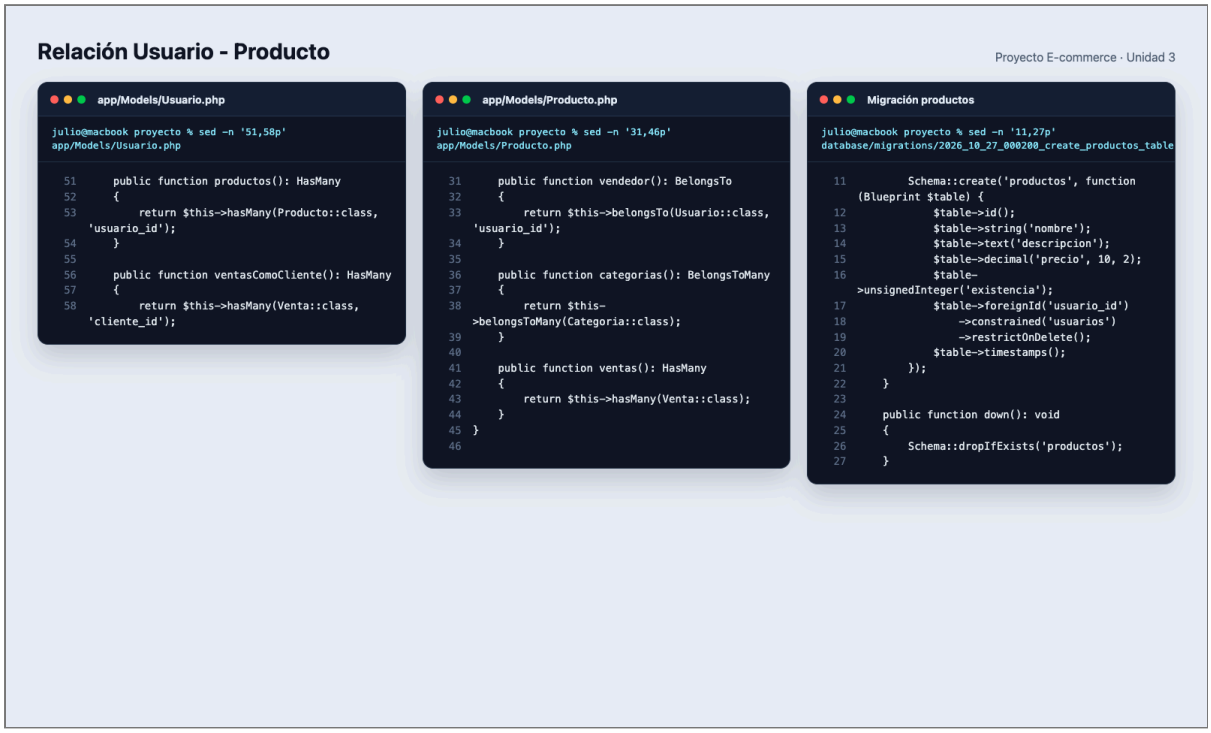


Figura 9. Relaciones en ambos modelos y llave foránea de la migración.

Descripción breve:

Producto::vendedor() pertenece a Usuario, y **Usuario::productos()** recupera los productos del vendedor. La llave foránea usa integridad restrictiva para evitar eliminar un usuario que todavía tenga productos relacionados.

EVIDENCIA 5 · SISTEMA

Producto con múltiples categorías

Responsable principal: Judith

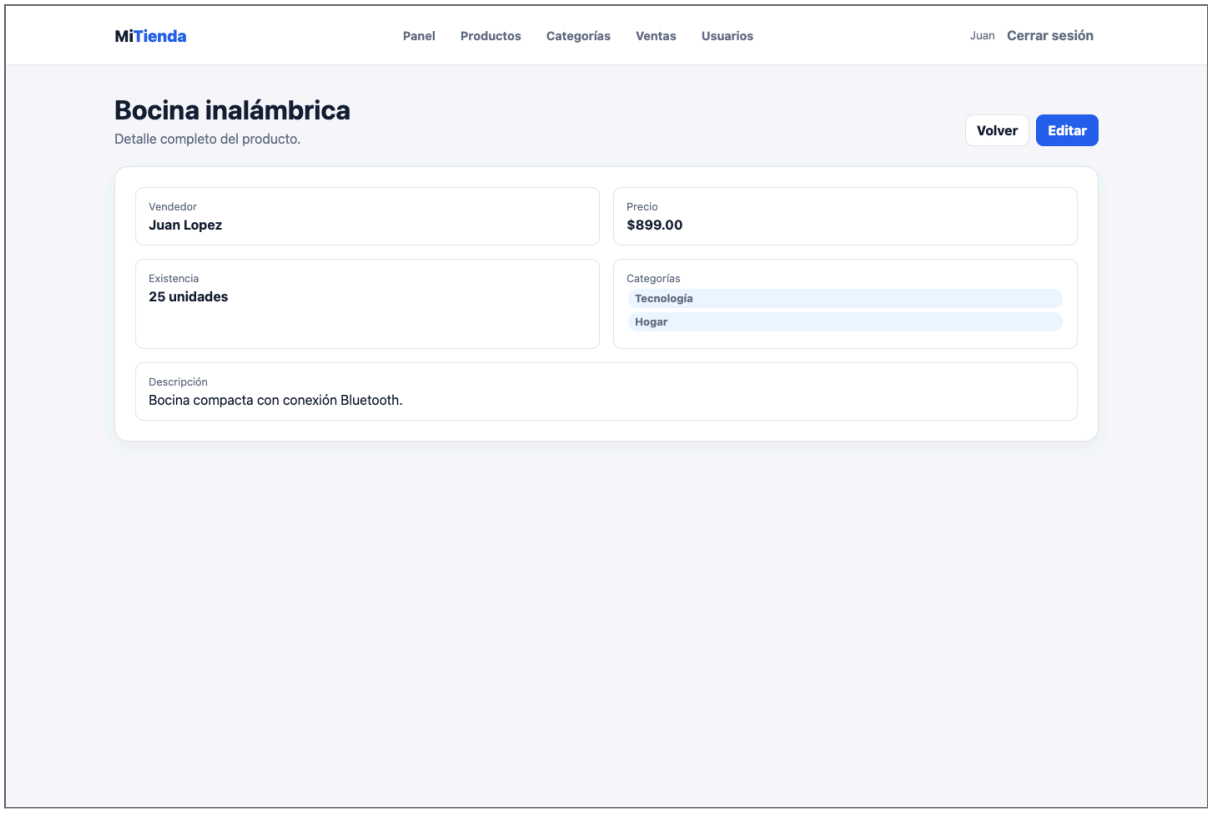


Figura 10. La bocina inalámbrica pertenece a Tecnología y Hogar.

Descripción breve:

La interfaz demuestra que un mismo producto puede vincularse con más de una categoría sin duplicar el producto. Las asociaciones se conservan en la tabla pivote.

EVIDENCIA 5 · SISTEMA

Categoría con sus productos

Responsable principal: Judith

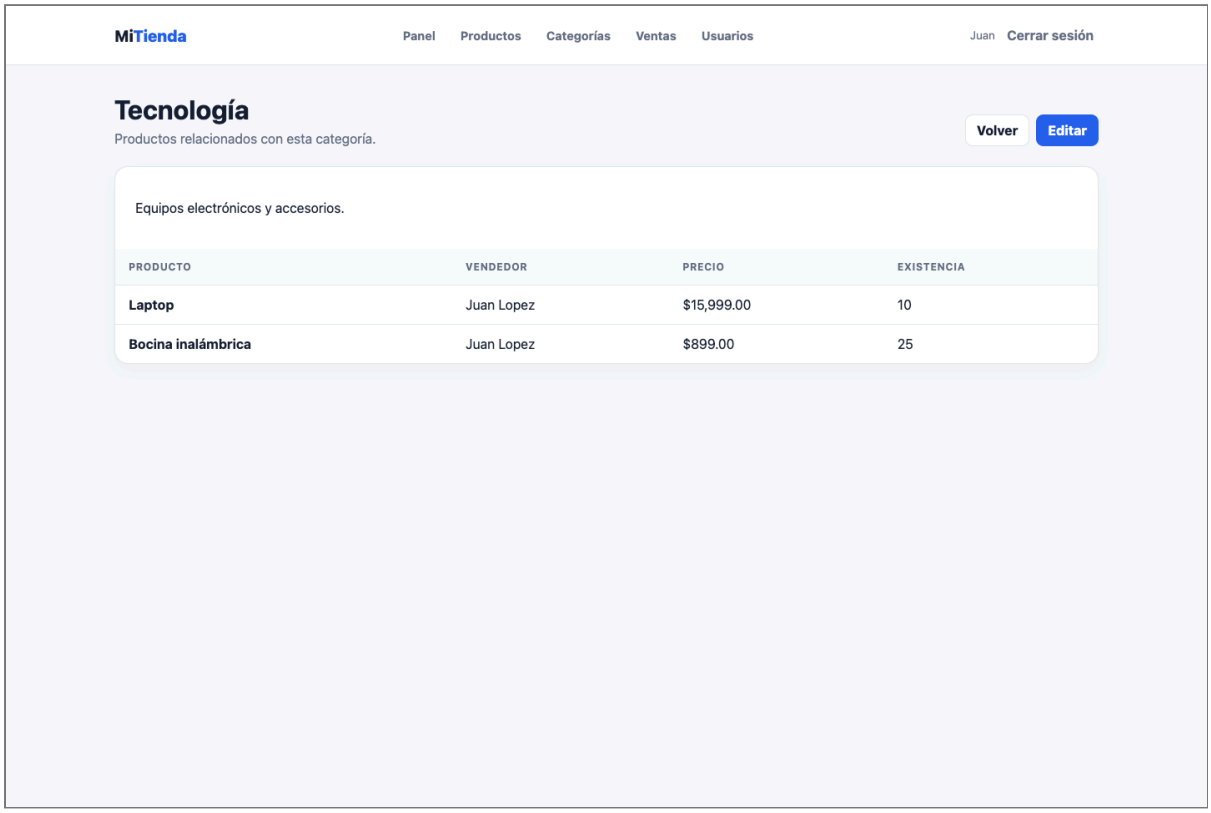


Figura 11. Recorrido inverso desde Tecnología hacia sus productos.

Descripción breve:

La relación es bidireccional: una categoría recupera todos sus productos y muestra también al vendedor de cada uno. Así se comprueba el lado inverso de la relación muchos a muchos.

EVIDENCIA 5 · CÓDIGO

belongsToMany y tabla pivote

Responsable principal: Judith

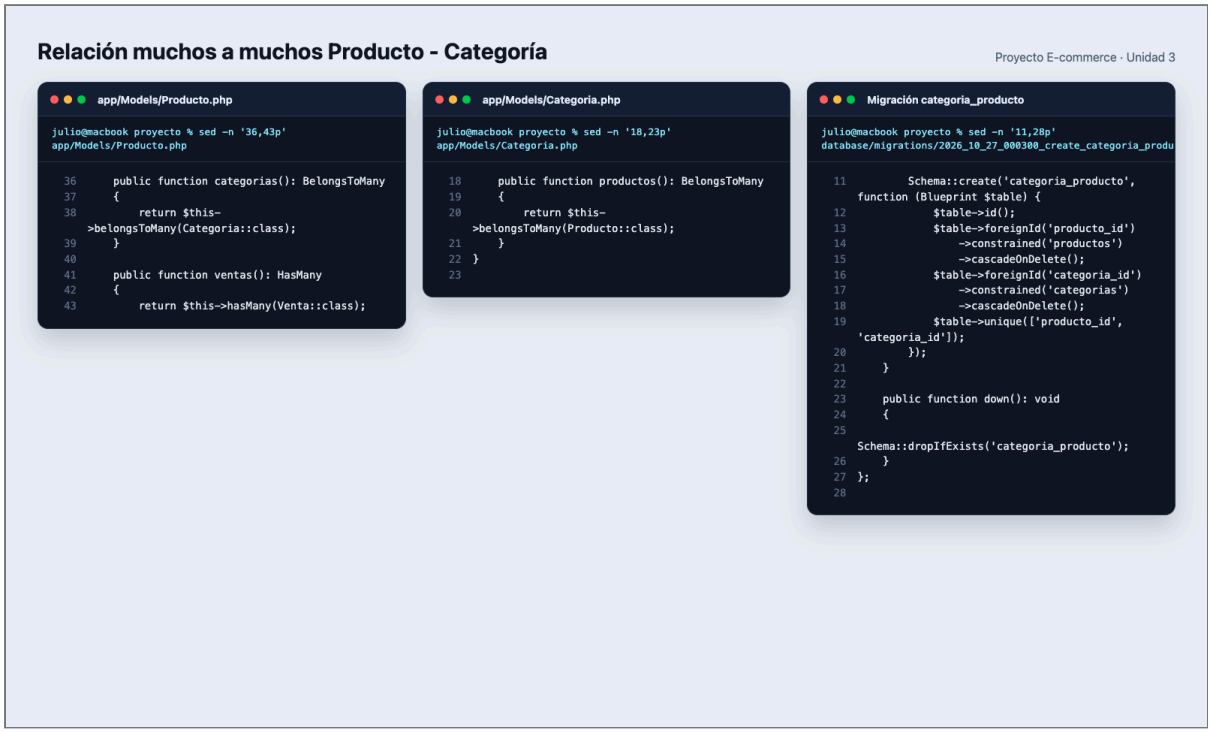


Figura 12. Relaciones bidireccionales y migración categoria_producto.

Descripción breve:

Producto y Categoria implementan **belongsToMany**. La tabla **categoria_producto** guarda ambas llaves foráneas, elimina asociaciones al borrar cualquiera de los registros y evita duplicados con un índice único compuesto.

EVIDENCIA 6 · SISTEMA

Venta registrada

Responsable principal: Julio

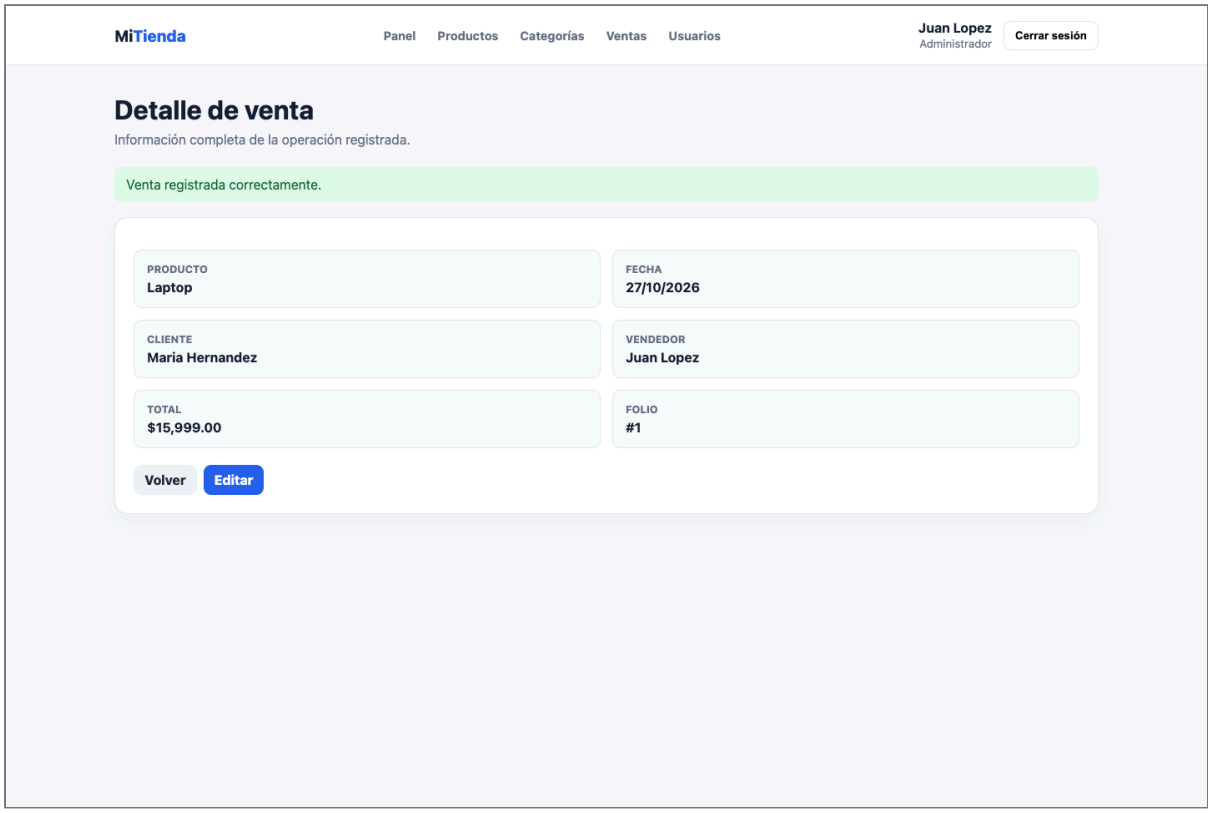


Figura 13. Detalle real con producto, cliente, vendedor, fecha, total y folio.

Descripción breve:

La operación distingue a Maria Hernandez como cliente y a Juan Lopez como vendedor de la Laptop. Al guardarse, la venta conserva las tres referencias y dispara el evento que escribe en **ventas.log**.

EVIDENCIA 6 · CÓDIGO

Relaciones de Venta

Responsable principal: Julio

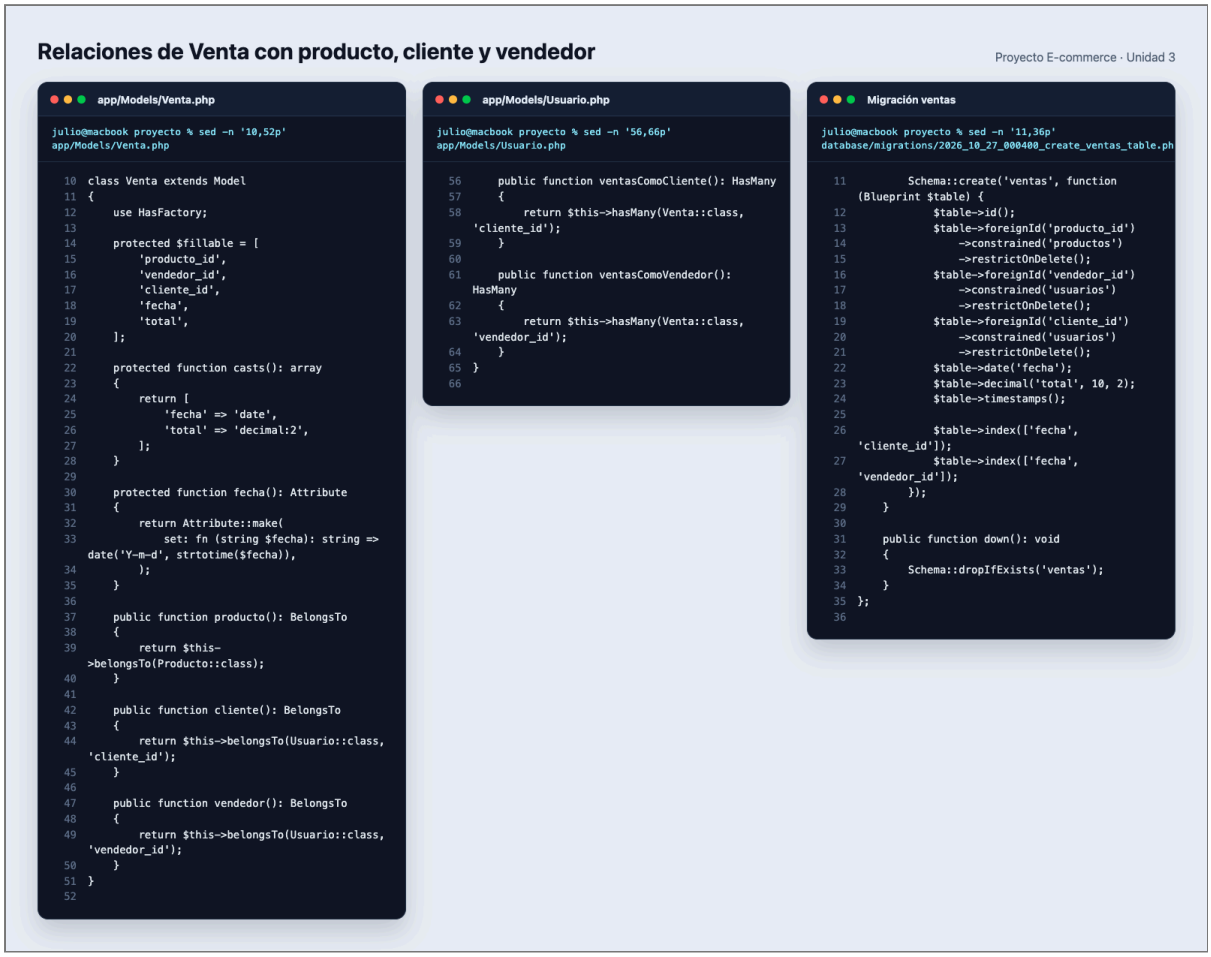


Figura 14. Producto, cliente y vendedor vinculados mediante relaciones Eloquent.

Descripción breve:

Venta pertenece a Producto y define dos relaciones distintas hacia Usuario usando **cliente_id** y **vendedor_id**. Usuario mantiene las relaciones inversas para consultar compras y ventas por separado.

EVIDENCIA 7 · SISTEMA

Errores de validación

Responsables: Judith y Julio

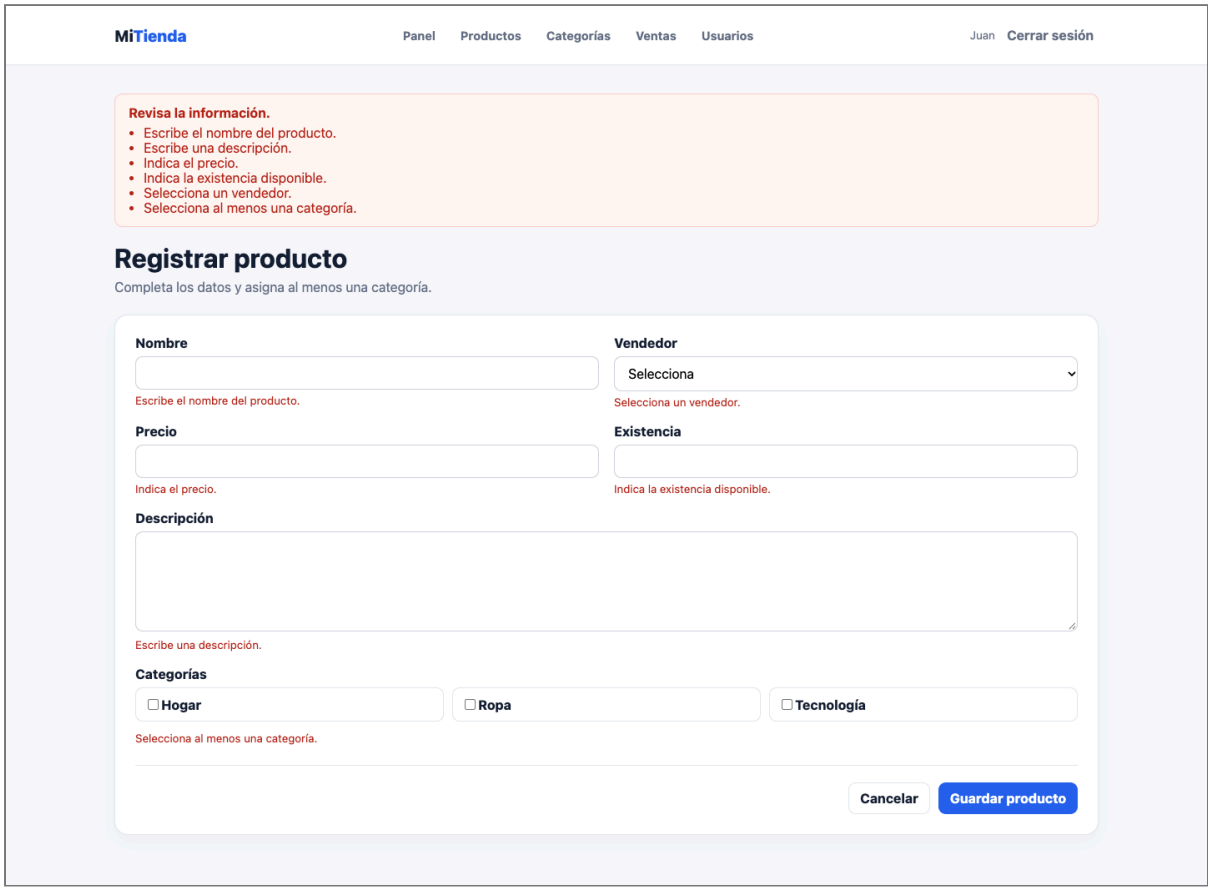


Figura 15. Formulario de producto rechazado por campos obligatorios.

Descripción breve:

Al enviar el formulario vacío, el servidor regresa mensajes específicos junto a cada campo y conserva al usuario en la pantalla de captura. Ningún registro incompleto llega al controlador.

EVIDENCIA 7 · CÓDIGO

Reglas en FormRequest

Responsables: Judith y Julio

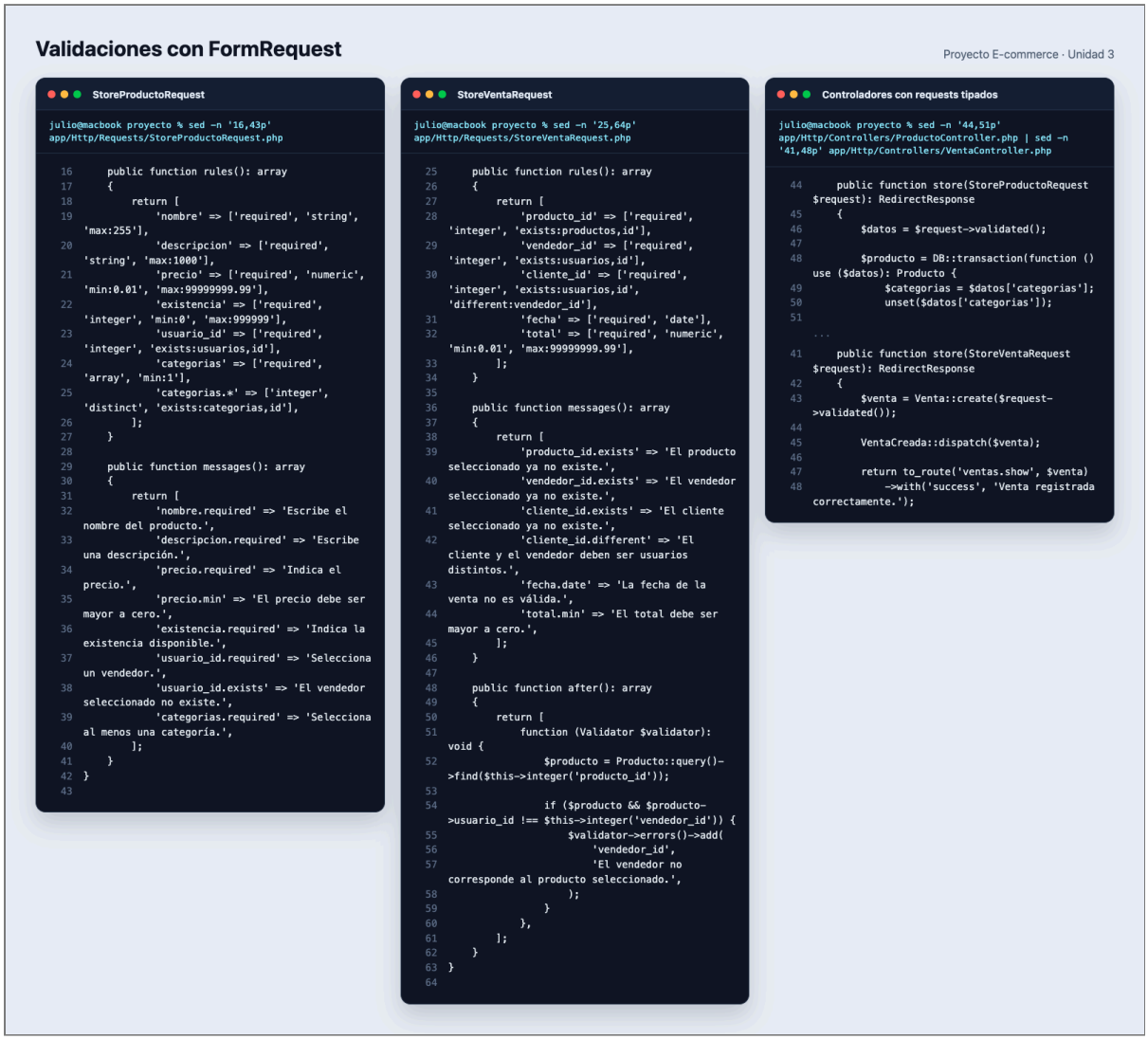


Figura 16. Reglas required, string, numeric, exists, min, max, integer, array, distinct y date.

Descripción breve:

Los FormRequest concentran autorización y validación antes de ejecutar el CRUD. Los controladores reciben solicitudes tipadas y trabajan exclusivamente con **validated()**, manteniendo separadas las reglas y la lógica de negocio.

EVIDENCIA 8 · SISTEMA

Datos sembrados

Responsables: Rodri y Judith

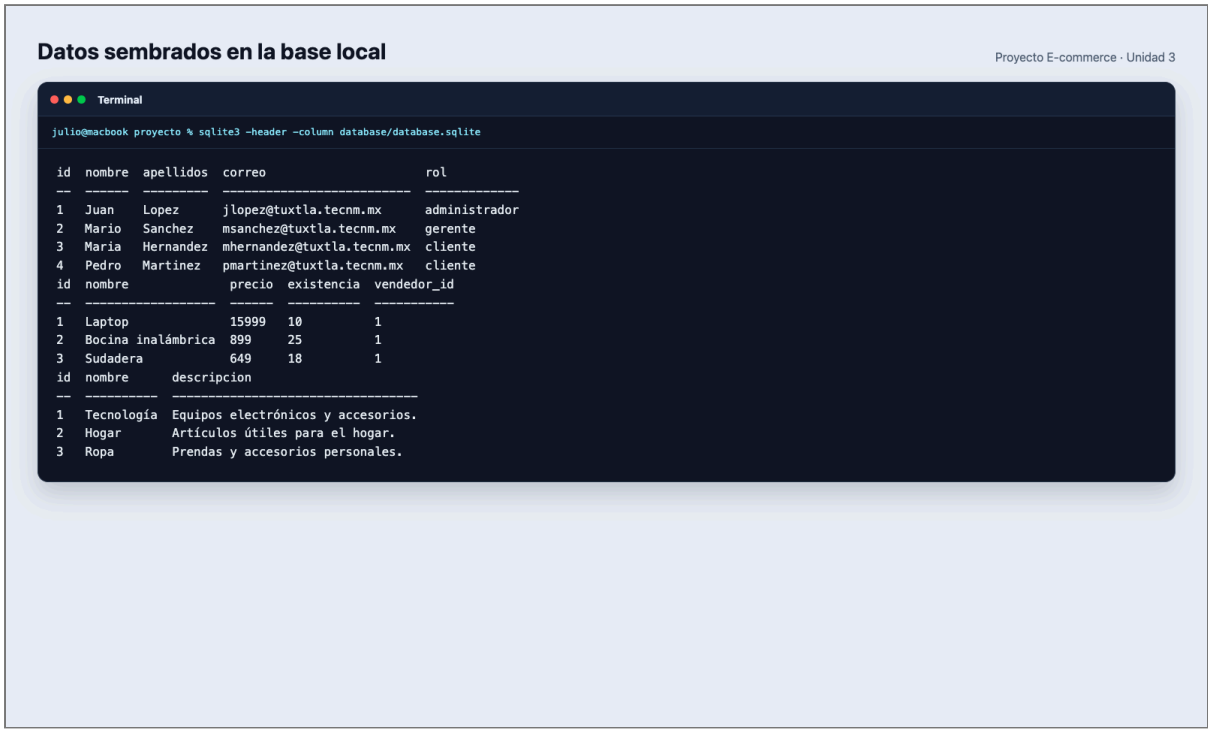


Figura 17. Consulta real de usuarios, productos y categorías después de migrate:fresh --seed.

Descripción breve:

DatabaseSeeder ejecuta los Seeders en el orden necesario: primero usuarios, luego categorías y finalmente productos. La captura muestra el resultado real; UsuarioFactory queda disponible para pruebas y generación controlada de cuentas.

EVIDENCIA 8 · CÓDIGO

UsuarioFactory

Responsable principal: Rodri

UsuarioFactory con datos y contraseña requeridos

Proyecto E-commerce · Unidad 3

database/factories/UsuarioFactory.php

```
julio@macbook proyecto % sed -n '14,72p' database/factories/UsuarioFactory.php
```

```php
class UsuarioFactory extends Factory
{
    protected $model = Usuario::class;

    public function definition(): array
    {
        $nombres = ['Juan', 'Mario', 'Maria', 'Pedro'];
        $apellidos = ['Lopez', 'Sanchez', 'Hernandez', 'Martinez'];
        $combinaciones = [];

        foreach ($nombres as $nombreDisponible) {
            foreach ($apellidos as $apellidoDisponible) {
                $combinaciones[] = [
                    'nombre' => $nombreDisponible,
                    'apellidos' => $apellidoDisponible,
                    'correo' => strtolower(Str::ascii(substr($nombreDisponible, 0, 1).$apellidoDisponible)).'@tuxtla.tecnm.mx',
                ];
            }
        }

        $correosUsados = Usuario::query()->pluck('correo')->all();
        $opciones = collect($combinaciones)
            ->shuffle()
            ->unique('correo')
            ->reject(fn (array $datos): bool => in_array($datos['correo'], $correosUsados, true))
            ->values();

        if ($opciones->isEmpty()) {
            throw new \RuntimeException('No quedan correos institucionales disponibles para la fábrica.');
        }

        $identidad = fake()->randomElement($opciones->all());

        return [
            'nombre' => $identidad['nombre'],
            'apellidos' => $identidad['apellidos'],
            'correo' => $identidad['correo'],
            'clave' => Hash::make('123'),
            'rol' => fake()->randomElement([UserRole::Client, UserRole::Manager]),
            'remember_token' => Str::random(10),
        ];
    }

    public function administrador(): static
    {
        return $this->state(fn () => ['rol' => UserRole::Administrator]);
    }

    public function gerente(): static
    {
        return $this->state(fn () => ['rol' => UserRole::Manager]);
    }

    public function cliente(): static
    {
        return $this->state(fn () => ['rol' => UserRole::Client]);
    }
}

```

Figura 18. Nombres y apellidos permitidos, correo institucional, clave 123 con Hash y roles.

Descripción breve:

La Factory limita nombres y apellidos a los valores indicados, forma el correo con la inicial y el apellido y evita colisiones entre Mario y Maria. La contraseña se genera con Hash y el rol se limita a Cliente o Gerente, con estados adicionales para las pruebas.

EVIDENCIA 8 · CÓDIGO

Seeders

Responsables: Rodri y Judith

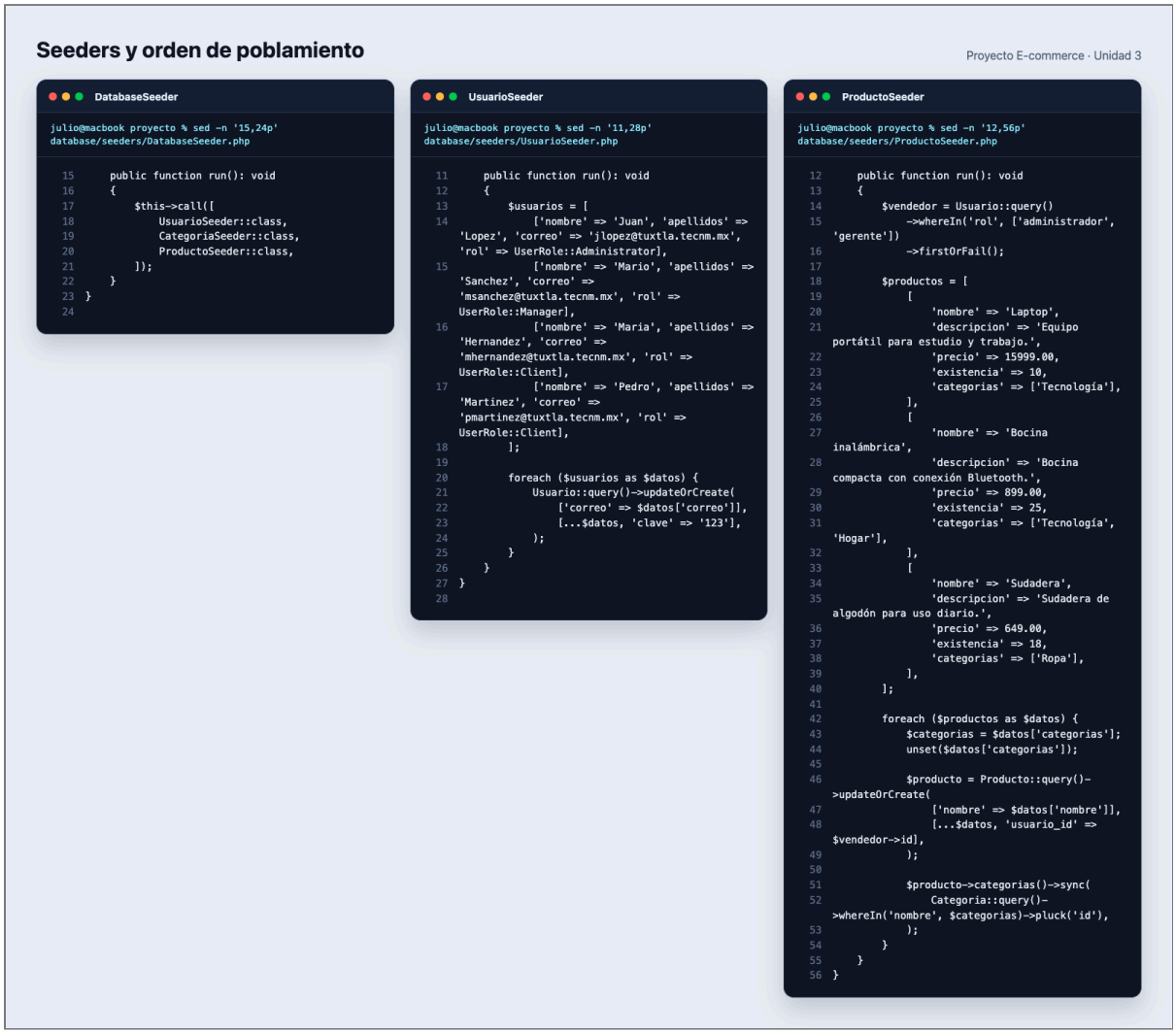


Figura 19. Orden de ejecución y creación de usuarios, categorías y productos relacionados.

Descripción breve:

Los Seeders usan updateOrCreate para poder ejecutarse más de una vez y sincronizan las categorías de cada producto. La clave 123 pasa por el cast hashed del modelo y nunca se guarda en texto plano.

EVIDENCIA 9 · SISTEMA

Bitácoras reales

Responsables: Rodri, Judith y Julio

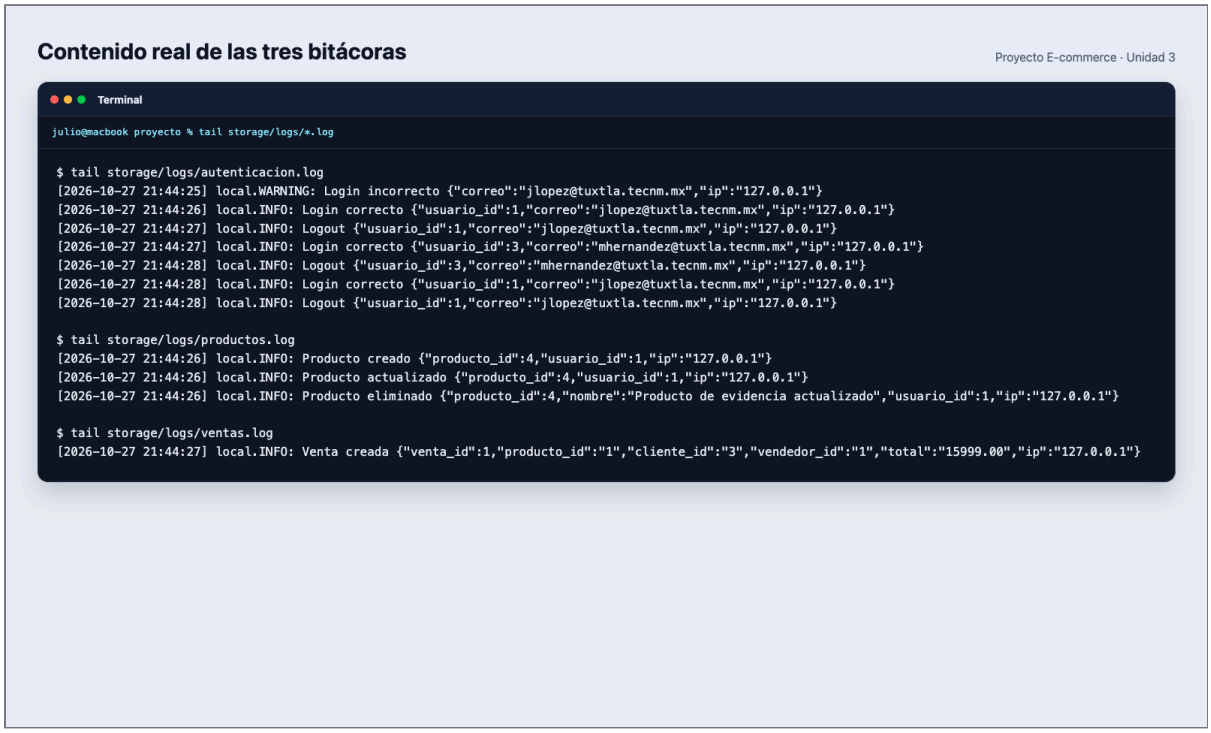


Figura 20. autenticacion.log, productos.log y ventas.log generados durante el recorrido del sistema.

Descripción breve:

Los registros muestran login correcto, login incorrecto, logout, creación, actualización y eliminación de producto, además de la venta creada. Cada canal guarda contexto acorde al evento, como usuario, correo, producto, venta e IP.

EVIDENCIA 9 · CÓDIGO

Canales y Log::channel

Responsables: Rodri, Judith y Julio



Figura 21. Configuración de canales y puntos donde se escriben los eventos.

Descripción breve:

config/logging.php dirige cada canal a su archivo. La autenticación y los productos escriben desde sus flujos correspondientes; VentaCreada se atiende mediante un listener descubierto por Laravel para registrar exactamente una entrada por operación.

EVIDENCIA 10 · SISTEMA

Llaves foráneas en SQLite

Responsables: Rodri, Judith y Julio

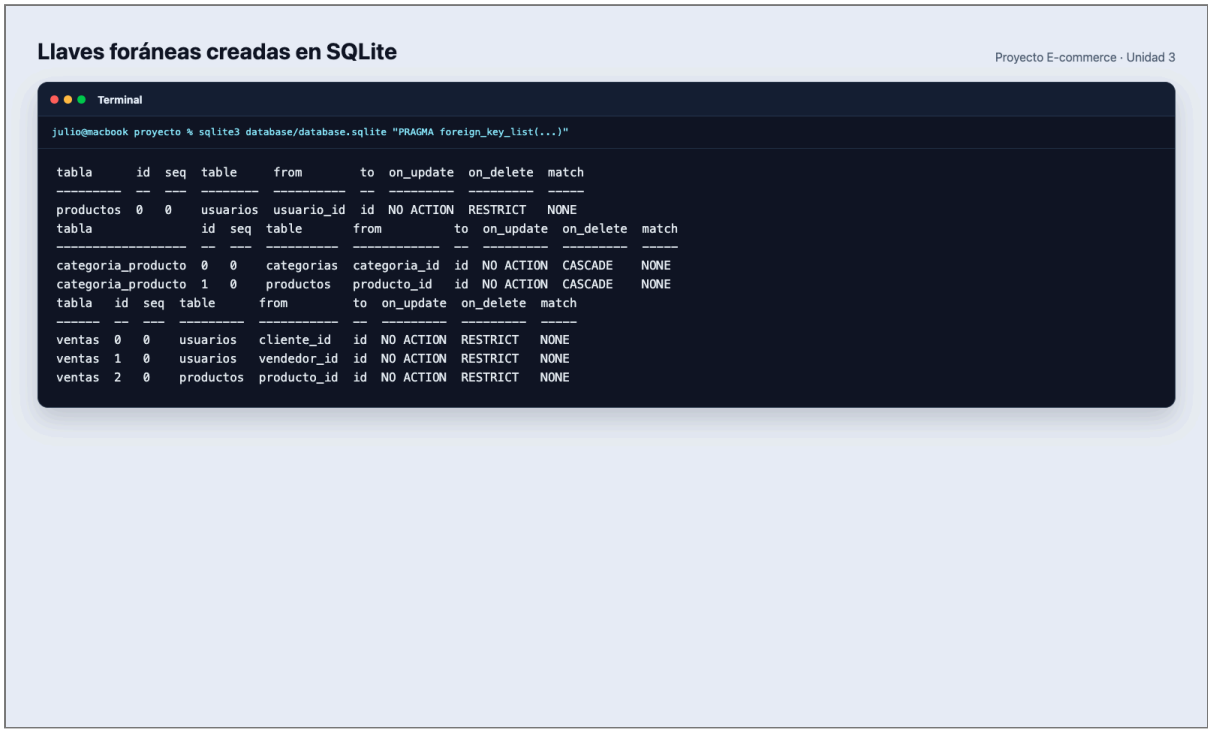


Figura 22. Inspección real de referencias y reglas ON DELETE.

Descripción breve:

La base contiene referencias de productos a usuarios, de la pivote a productos y categorías, y de ventas a producto, cliente y vendedor. Las reglas cascade y restrict protegen la integridad de acuerdo con cada relación.

EVIDENCIA 10 · CÓDIGO

Migraciones

Responsables: Rodri, Judith y Julio

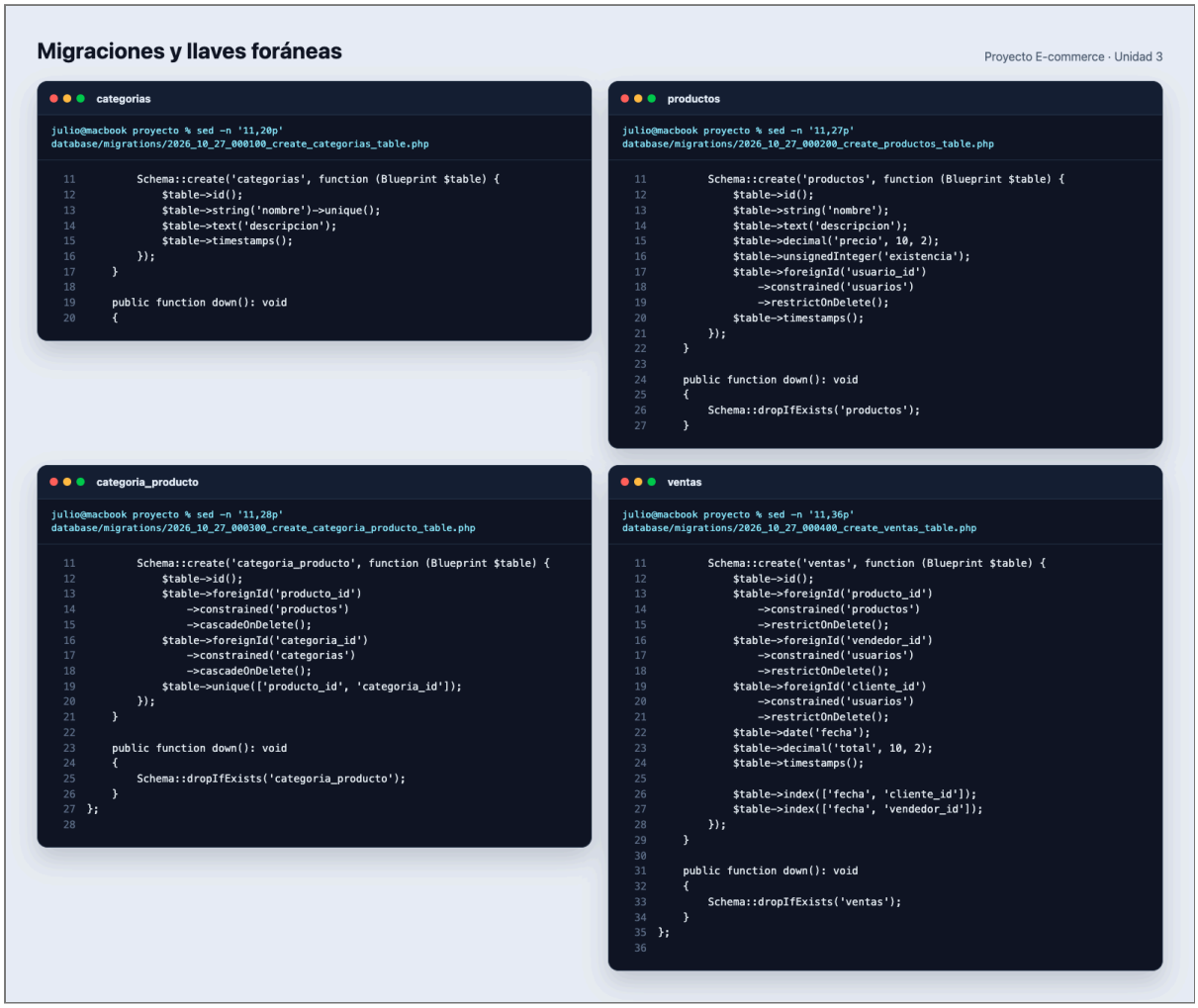


Figura 23. Migraciones de categorías, productos, tabla pivote y ventas.

Descripción breve:

Las migraciones definen los tipos, índices, claves únicas y relaciones del sistema. La instalación fue comprobada con **php artisan migrate:fresh --seed** sobre una base SQLite nueva.

REQUISITO ADICIONAL · SISTEMA

Categorías obtenidas por usuario

Responsable de integración: Julio



Figura 24. Consulta real de las categorías de los productos vendidos por Juan Lopez.

Descripción breve:

La consulta devuelve Tecnología, Hogar y Ropa sin duplicados. El recorrido comienza en Usuario, atraviesa sus productos y usa las asignaciones de la tabla pivote para recuperar las categorías.

REQUISITO ADICIONAL · CÓDIGO

Solución hasManyThrough con pivote

Responsable de integración: Julio

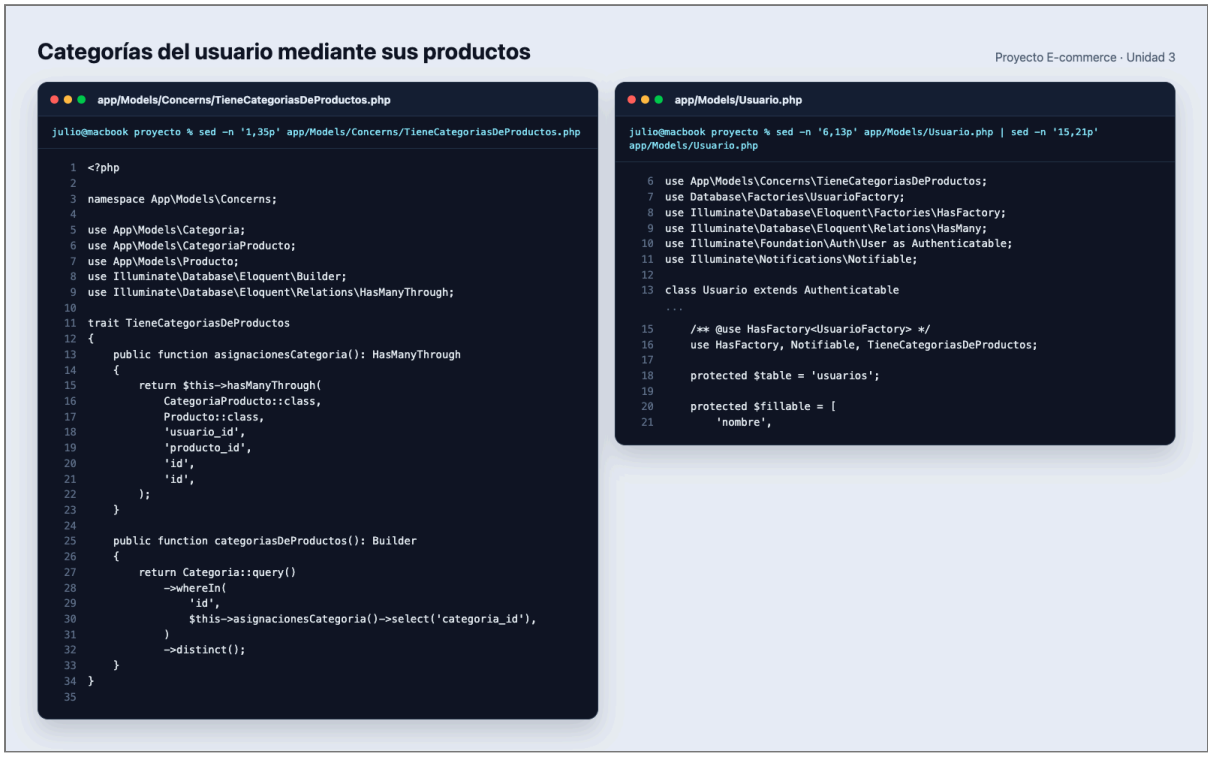


Figura 25. Trait usado por Usuario para atravesar Producto y CategoriaProducto.

Descripción breve:

Como Producto–Categoría es muchos a muchos, no existe un hasManyThrough directo hacia Categoria. La solución usa **hasManyThrough** para llegar a las asignaciones CategoriaProducto y después obtiene las categorías con una consulta distinct, respetando la pivote y evitando duplicados.

ANEXO · CÓDIGO

Rutas y registro de Policies

Responsable de integración: Julio

Rutas, Policies y evento registrados

Proyecto E-commerce · Unidad 3

routes/web.php

```
julio@macbook proyecto % sed -n '1,40p' routes/web.php
```

```php
<?php

use App\Enums\UserRole;
use App\Http\Controllers\CategoriaController;
use App\Http\Controllers\DashboardRedirectController;
use App\Http\Controllers\ProductoController;
use App\Http\Controllers\UsuarioController;
use App\Http\Controllers\VentaController;
use Illuminate\Support\Facades\Route;

Route::get('/', function () {
    return view('welcome');
})->name('home');

Route::view('/quienes-somos', 'publica.nosotros')->name('nosotros');
Route::view('/contacto', 'publica.contacto')->name('contacto');

Route::middleware('auth')->group(function () {
    Route::get('/dashboard', DashboardRedirectController::class)->name('dashboard');

    Route::view('/cliente/dashboard', 'dashboards.cliente')
        ->middleware('role:'.UserRole::Client->value)
        ->name('cliente.dashboard');

    Route::view('/administrador/dashboard', 'dashboards.administrador')
        ->middleware('role:'.UserRole::Administrator->value)
        ->name('administrador.dashboard');

    Route::view('/gerente/dashboard', 'dashboards.gerente')
        ->middleware('role:'.UserRole::Manager->value)
        ->name('gerente.dashboard');

    Route::resource('productos', ProductoController::class);
    Route::resource('categorias', CategoriaController::class);
    Route::resource('ventas', VentaController::class);
    Route::resource('usuarios', UsuarioController::class);
});

require __DIR__.'/auth.php';

```

app/Providers/AppServiceProvider.php

```
julio@macbook proyecto % sed -n '26,34p' app/Providers/AppServiceProvider.php
```

```php
    public function boot(): void
    {
        Gate::policy(Usuario::class, UsuarioPolicy::class);
        Gate::policy(Producto::class, ProductoPolicy::class);
        Gate::policy(Categoria::class, CategoriaPolicy::class);
        Gate::policy(Venta::class, VentaPolicy::class);
    }
}

```

Figura 26. Rutas protegidas y asociación explícita entre modelos y Policies.

Descripción breve:

Las rutas de los cuatro CRUD quedan bajo middleware auth. Los paneles aplican middleware por rol y AppServiceProvider registra las Policies de Usuario, Producto, Categoria y Venta.

ANEXO · SISTEMA

Rutas registradas

Responsable de integración: Julio

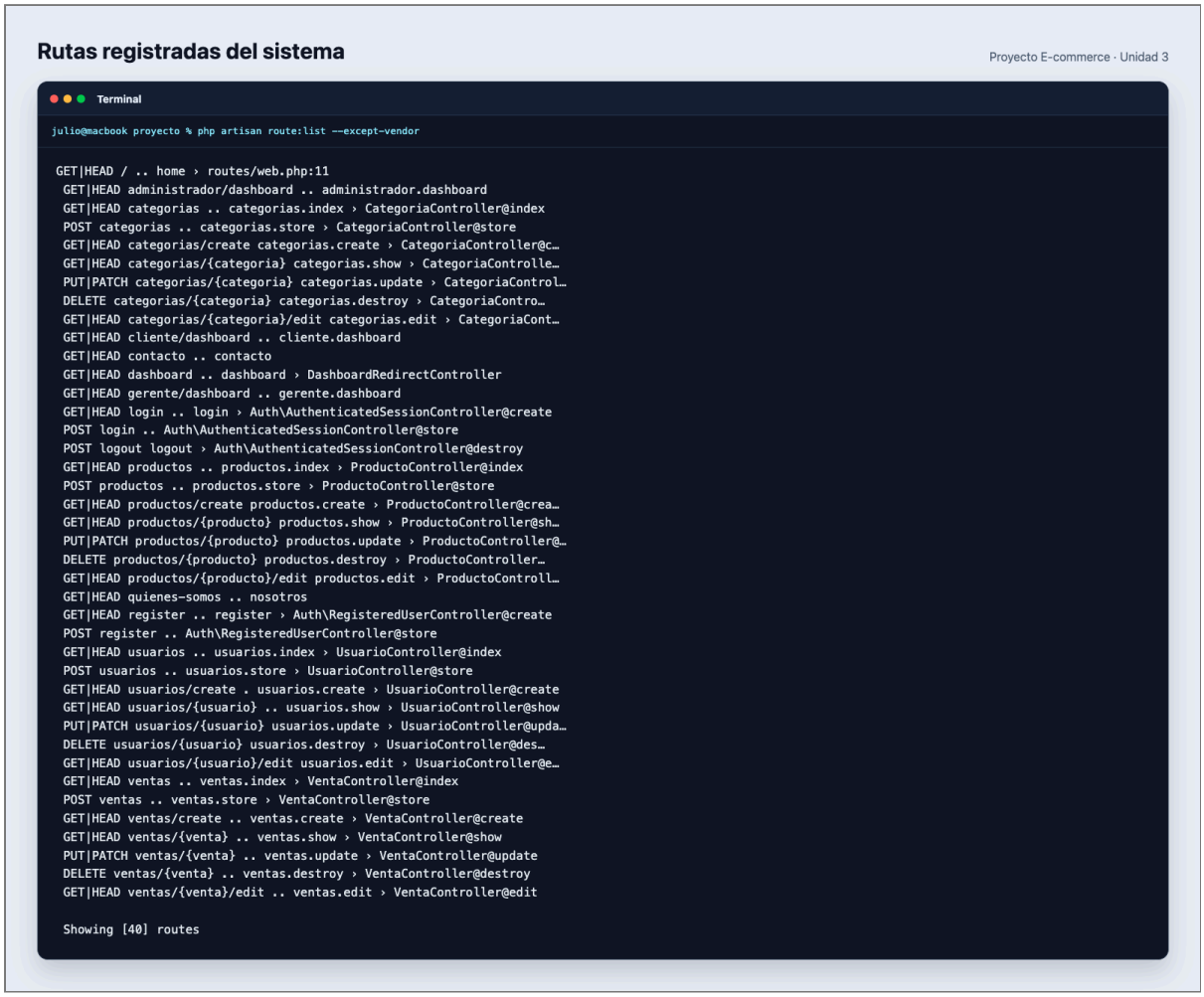


Figura 27. Salida real de php artisan route:list --except-vendor.

Descripción breve:

La lista confirma las rutas públicas, login, logout, paneles por rol y recursos completos de Usuarios, Productos, Categorías y Ventas.

REVISIÓN FINAL

Lista de verificación

Requisito	Estado
Usuario hereda de Authenticatable y provider configurado	**Cumplido**
Login manual correcto, incorrecto y logout	**Cumplido**
Roles Administrador, Gerente y Cliente	**Cumplido**
Policies y Gates en los CRUD	**Cumplido**
CRUD completo de Usuarios, Productos, Categorías y Ventas	**Cumplido**
Relaciones bidireccionales y tabla pivote	**Cumplido**
FormRequest para altas y actualizaciones	**Cumplido**
UsuarioFactory, CategoriaSeeder y ProductoSeeder	**Cumplido**
Tres canales de bitácora con contenido real	**Cumplido**
Relación de categorías mediante productos	**Cumplido**
Migraciones y llaves foráneas	**Cumplido**
Pruebas automatizadas y recorrido en navegador	**Cumplido**

Resultado de verificación

Pruebas automatizadas 45 correctas y 156 aserciones.

Base de datos Migraciones y Seeders ejecutados desde cero.

Interfaz Menú completo y vistas responsivas para cada módulo.

Bitácoras Tres archivos separados con eventos reales.

Documento preparado con una captura por página para conservar la legibilidad del código y de la ejecución.